

Straight Lines and Bends

Linear functions, and why networks need a bend · video 1:49

This lesson has one big idea: **linear layers alone are not enough**. A matrix can stretch and rotate, but a stack of matrices is always just one more matrix. Putting a simple **bend** between the layers, a nonlinear function like ReLU or GELU, is what lets a neural network build curved, complicated functions.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 10).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. Linear: double in, double out	▶ 0:08-0:27	4	☐
2. Linear layers collapse into one	▶ 0:28-0:38	4	☐
3. What a straight map can't do	▶ 0:38-0:50	3	☐
4. The fix: a bend called ReLU	▶ 0:51-1:08	4	☐
5. GELU: a smooth ReLU	▶ 1:08-1:19	4	☐
6. Curves from many straight pieces	▶ 1:20-1:27	3	☐
7. A layer in code	▶ 1:28-1:47	3	☐

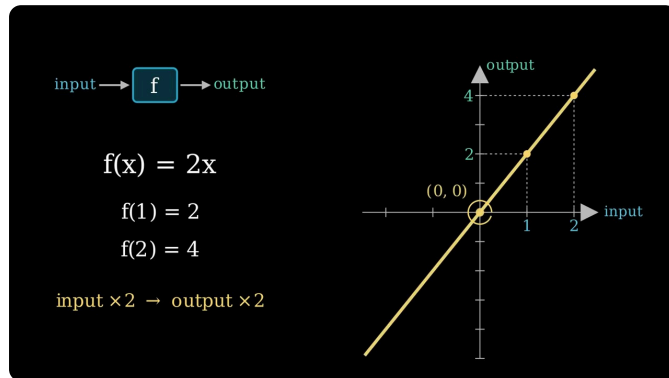
Before you start: this lesson uses vectors (F01) and multiplying a matrix by a vector or by another matrix, including NumPy's @ (F03). You will meet the ideas again in episode 8 (The MLP: Where Facts Live), whose layer is exactly “expand, bend with GELU, project back”, and in episode 9 (The Transformer Block), where those layers are stacked.

CONCEPT 1 OF 7

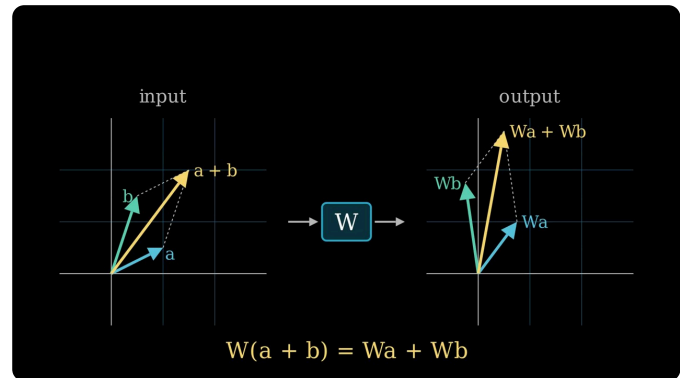
Linear: double in, double out

▶ Video 0:08-0:27

A function takes an input and gives an output. The simplest kind is **linear**, like $f(x) = 2x$: $f(1) = 2$ and $f(2) = 4$, so doubling the input doubles the output. Its graph is a **straight line through the origin**. Multiplying a vector by a **matrix** W is linear too. **Scale** the input and the output scales the same way, $W(2a) = 2 \cdot Wa$; **add** two inputs and their outputs add, $W(a + b) = Wa + Wb$.



▶ 0:17 $f(x) = 2x$: a straight line through the origin.



▶ 0:27 A matrix is linear too: $W(a + b) = Wa + Wb$.

KEY IDEA

Linear means scaling and adding pass straight through: $f(2x) = 2 \cdot f(x)$ and $f(a + b) = f(a) + f(b)$. Every matrix is linear.

Check 1: Linear: double in, double out

Answer all 4 questions before looking at the answers. Pass mark: all correct.

1.1 MULTIPLE CHOICE

Which of these functions is linear?

- A. $f(x) = 2x + 1$
- B. $f(x) = -3x$
- C. $f(x) = x^2$
- D. $f(x) = 10$

1.2 CALCULATION

The video's matrix W has rows $(1, -0.5)$ and $(0.5, 1)$, and the vectors are $a = (1, 0.5)$ and $b = (0.5, 1.5)$. Compute Wa , Wb and $W(a + b)$. Does $W(a + b) = Wa + Wb$?

.....

.....

1.3 CALCULATION

You know $Wa = (0.75, 1)$. Without multiplying by W again, write down $W(2a)$ and $W(-a)$.

.....

1.4 TRUE OR FALSE

"The graph of a linear function such as $f(x) = 5x$ always passes through $(0, 0)$."

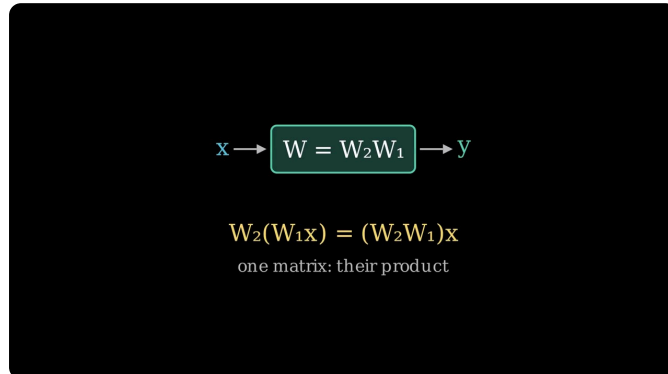
True ☐ False ☐

CONCEPT 2 OF 7

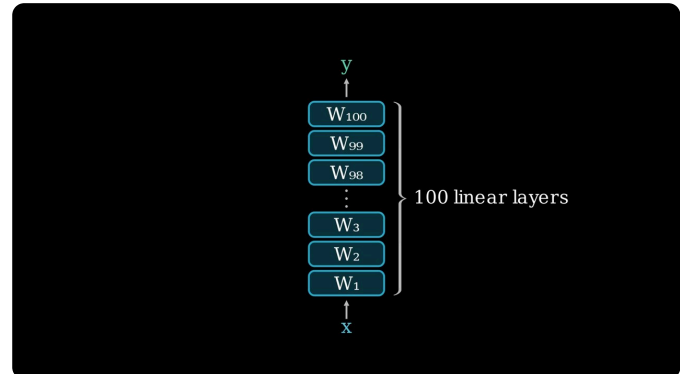
Linear layers collapse into one

▶ Video 0:28–0:38

Here's the catch. Apply W_1 to x , then W_2 , and you get exactly what one **single** matrix gives, their product: $W_2(W_1x) = (W_2W_1)x$. Stack a hundred linear layers and they still **collapse** into one matrix, $W_{100}\cdots W_2W_1$.



▶ 0:34 W_1 then W_2 is one matrix: their product.



▶ 0:36 100 linear layers: still one matrix.

KEY IDEA

Linear after linear is still linear. With nothing in between, a stack of matrices is **no more powerful than one**.

Check 2: Linear layers collapse into one

Answer all 4 questions before looking at the answers. Pass mark: all correct.

2.1 CALCULATION

W_1 has rows (1, 2) and (0, 1); W_2 has rows (2, 0) and (1, 1); $x = (1, 1)$. (a) Compute W_1x , then $W_2(W_1x)$. (b) Compute the product W_2W_1 , then $(W_2W_1)x$.

2.2 CALCULATION

Three layers act on single numbers: multiply by 3, then by -2 , then by 0.5. What single layer does the same?

2.3 MULTIPLE CHOICE

A network has 100 linear layers and no bends. How many matrices do you need to compute exactly the same outputs?

- A. 100, one per layer
- B. 50, since layers merge in pairs
- C. 1
- D. It depends on the input

2.4 TRUE OR FALSE

"Adding more linear layers, with nothing between them, lets a network represent more complicated functions."

True ☐ False ☐

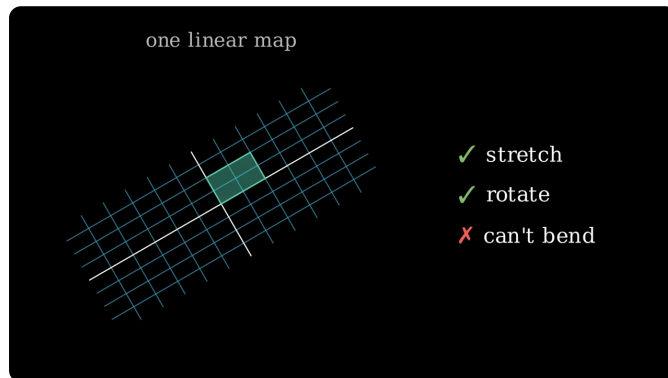
CONCEPT 3 OF 7

What a straight map can't do

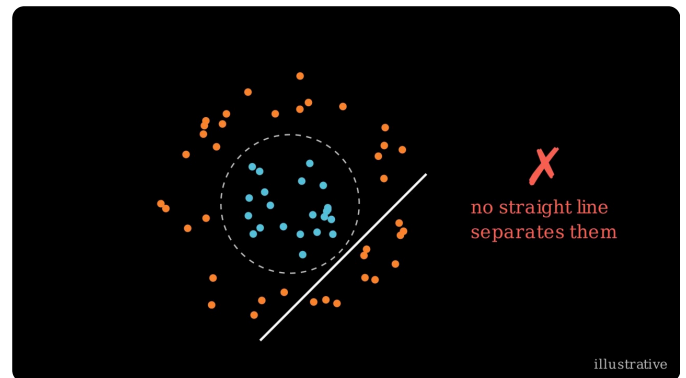
▶ Video 0:38-0:50

A single linear map can only do so much. It can **stretch** and **rotate** space, but it **can't bend** it: straight grid lines stay straight.

That limits what it can decide. The blue points lie inside a circle and the orange points around it. Try any straight line: orange points end up on **both** sides, so no straight line separates inside from outside. The boundary we need is curved, and a stack of linear layers can't make one either (concept 2).



▶ 0:45 One linear map can stretch and rotate the grid, but straight lines stay straight: it can't bend.



▶ 0:50 Points inside the circle vs outside it: every straight line leaves orange points on both sides.

KEY IDEA

Linear maps only make **straight** things: straight lines, flat boundaries. Problems like “inside vs outside a circle” need a **curve**.

Check 3: What a straight map can't do

Answer all 3 questions before looking at the answers. Pass mark: all correct.

3.1 MULTIPLE CHOICE

Which of these can a single matrix **not** do to the grid?

- A. Stretch it
- B. Rotate it
- C. Shear it (F03)
- D. Bend straight lines into curves

3.2 SHORT ANSWER

A one-dimensional version of the circle: the “inside” points are -0.5 and 0.5 , the “outside” points are -2 and 2 . Can one cut “ $x > c$ ” put both outside points on one side and both inside points on the other? Explain.

3.3 TRUE OR FALSE

“One linear layer can't separate the circle, but 50 linear layers stacked together can.”

True ☐ False ☐

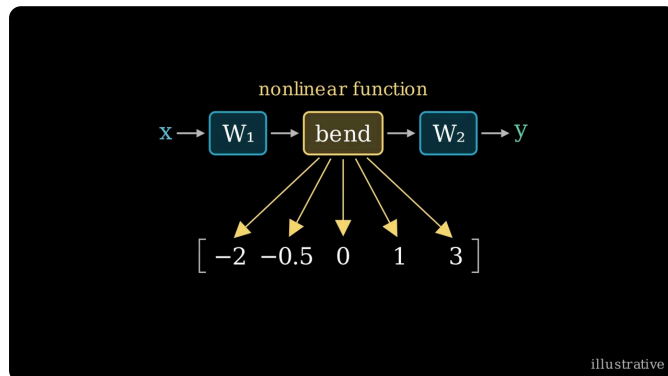
CONCEPT 4 OF 7

The fix: a bend called ReLU

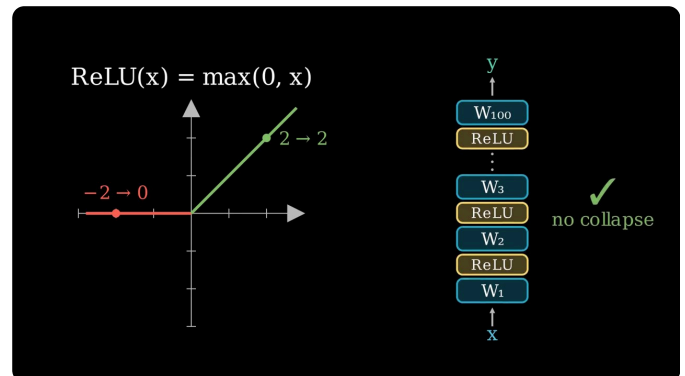
▶ Video 0:51-1:08

The fix is to add a **bend** between the layers: a **nonlinear** function, applied to **every number separately**. The layer becomes $x \rightarrow W_1 \rightarrow \text{bend} \rightarrow W_2 \rightarrow y$.

The simplest bend is **ReLU**, $\text{ReLU}(x) = \max(0, x)$. It keeps positive numbers and replaces negative ones with zero, so the vector $[-2, -0.5, 0, 1, 3]$ becomes **[0, 0, 0, 1, 3]**. A tiny change, but ReLU is not linear, so $W_2 \cdot \text{ReLU}(W_1 x)$ is no longer one matrix, and stacked layers can't collapse.



▶ 0:57 The bend sits between the layers and acts on every number separately.



▶ 1:07 $\text{ReLU}(x) = \max(0, x)$. With ReLU between them, stacked layers can't collapse.

KEY IDEA

A **nonlinearity** (bend) between linear layers stops them collapsing. ReLU = $\max(0, x)$, applied to each number on its own, is the simplest one.

Check 4: The fix: a bend called ReLU

Answer all 4 questions before looking at the answers. Pass mark: all correct.

4.1 CALCULATION

Apply ReLU to the vector $[4, -1, 0.5, -3]$.

4.2 CALCULATION

Check that ReLU breaks the "adding" rule of concept 1: compute $\text{ReLU}(3 + (-2))$ and $\text{ReLU}(3) + \text{ReLU}(-2)$.

4.3 PUT IN ORDER

Put one bent layer in the order the data flows: $y \cdot W_2 \cdot x \cdot \text{ReLU} \cdot W_1$.

4.4 TRUE OR FALSE

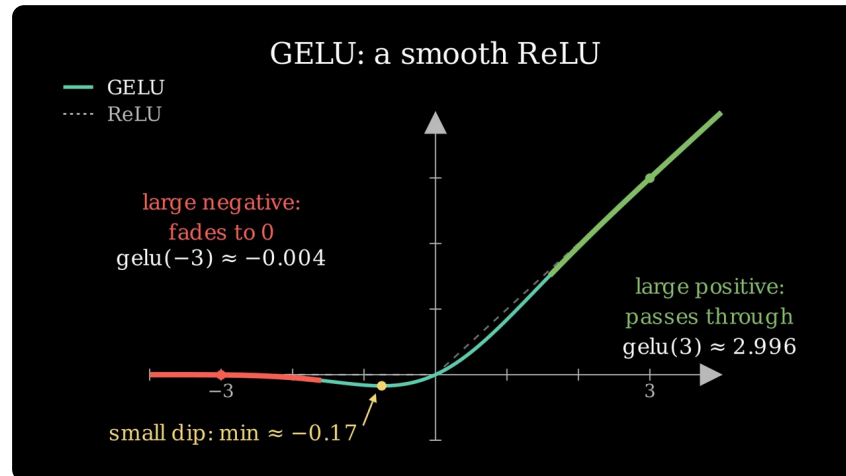
"ReLU mixes the numbers of a vector: each output number depends on all the input numbers."

True ☐ False ☐

GELU: a smooth ReLU

▶ Video 1:08–1:19

Transformers usually use **GELU**, a smooth version of ReLU. Far from zero the two agree: large positive inputs **pass through** ($\text{gelu}(3) \approx 2.996$) and large negative ones **fade to zero** ($\text{gelu}(-3) \approx -0.004$). Near zero, instead of ReLU's sharp corner, there is a **gentle curve**, with a small dip below zero down to about -0.17 .



▶ 1:19 GELU (teal) against ReLU (dashed): the same far from 0, but smooth near 0, with a small dip below zero.

KEY IDEA

$\text{GELU} \approx \text{ReLU}$ for large inputs, but it is **smooth** (no corner) around 0 and dips slightly below zero (minimum ≈ -0.17). It is still a bend, applied to every number separately.

Check 5: GELU: a smooth ReLU

Answer all 4 questions before looking at the answers. Pass mark: all correct.

5.1 CALCULATION

How far is GELU from ReLU at the video's two points? Compute $\text{relu}(3) - \text{gelu}(3)$ and $\text{relu}(-3) - \text{gelu}(-3)$.

5.2 MULTIPLE CHOICE

Given $\text{gelu}(1) \approx 0.841$ and $\text{gelu}(-1) \approx -0.159$, where do ReLU and GELU differ most?

- A. For large positive inputs, like 3
- B. For large negative inputs, like -3
- C. Near zero, around -1 to 1
- D. Nowhere: they are the same function

5.3 TRUE OR FALSE

"GELU never outputs a negative number."

True ☐ False ☐

5.4 SHORT ANSWER

A GELU receives the vector $[-3, 0, 3]$. Roughly what comes out?

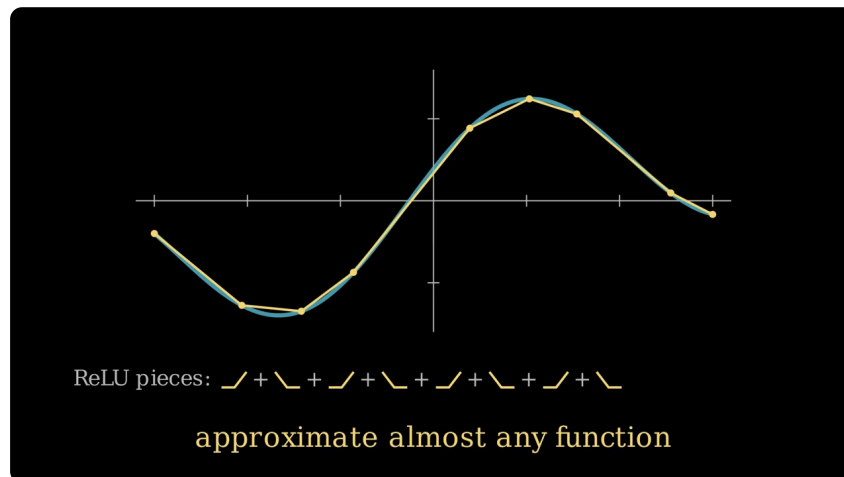
CONCEPT 6 OF 7

Curves from many straight pieces

▶ Video 1:20–1:27

Each ReLU adds one **hinge**: flat on one side, sloped on the other. Add several hinges at different places, with different slopes, and you get a line with **corners**. With enough corners, the straight pieces can follow a curve very closely.

That is what bends between the layers buy. A network can build **curved shapes out of many simple pieces**, and approximate almost any function. Without the bends, all it could ever make is one straight line.



▶ 1:27 A curve (blue) approximated by adding up ReLU pieces: the yellow bent line follows it closely.

KEY IDEA

ReLU hinges are simple pieces. Adding many of them gives bent lines that can follow **almost any curve**.

Check 6: Curves from many straight pieces

Answer all 3 questions before looking at the answers. Pass mark: all correct.

6.1 CALCULATION

Let $g(x) = \text{relu}(x - 1) + \text{relu}(-x - 1)$. Compute g at $x = -2, -0.5, 0.5$ and 2 . Does it separate the inside and outside points of exercise 3.2?

.....

.....

6.2 CALCULATION

Let $h(x) = \text{relu}(x) - 2 \cdot \text{relu}(x - 1)$. Compute $h(0), h(1), h(2)$ and $h(3)$. Describe the shape.

.....

6.3 MULTIPLE CHOICE

You add up several linear functions with no bends, such as $2x$ and $-3x$. What shape is the result?

- A. A curve
- B. A straight line
- C. A hinge with one corner
- D. A circle

CONCEPT 7 OF 7

A layer in code

▶ Video 1:28-1:47

In NumPy, ReLU is one line: `np.maximum(0, x)` compares every number with 0 and keeps the larger. A layer is a **matrix multiplication, then the bend, then the next matrix**. (In code the vector is written first, `x @ W1`; it is still one matrix multiplication.)

```
1 def relu(x):  
2     return np.maximum(0, x)  
3  
4 relu(np.array([-2, -0.5, 0, 1, 3])) # [0, 0, 0, 1, 3]  
5  
6 h = relu(x @ W1) # linear, then bend  
7 y = h @ W2      # linear again
```

▶ 1:34 The code from the video: ReLU in one line, then a layer: matrix, bend, matrix.

```
def relu(x):  
    return np.maximum(0, x)  
  
relu(np.array([-2, -0.5, 0, 1, 3])) # [0, 0, 0, 1, 3]  
  
h = relu(x @ W1) # linear, then bend  
y = h @ W2      # linear again
```

This is the MLP of episode 8: **expand** ($W1$ makes the vector longer), **bend with GELU**, and **project back** ($W2$ returns it to its original size). Episode 9 stacks these blocks.

KEY IDEA

One layer = `relu(x @ W1) @ W2`: linear, bend, linear. Remove the bend and the two matrices collapse into one, `W1 @ W2`.

Check 7: A layer in code

Answer all 3 questions before looking at the answers. Pass mark: all correct.

7.1 SHORT ANSWER

Remove the bend: `y = (x @ W1) @ W2`. Which single matrix W gives the same y for every x ?

7.2 PUT IN ORDER

Put the steps of episode 8's MLP in order: *project back · bend with GELU · expand*.

7.3 HANDS-ON CODE

Try it yourself. Here is a tiny layer: W1 expands 2 numbers to 3, W2 projects back to 2. (a) **By hand first:** what are `x @ W1`, `h` and `y`? Then run the code to check. (b) What does the last line print, and why are its two arrays equal? (c) Why is `y` different from `y_lin`? Which number in `x @ W1` made the difference?

```
import numpy as np

def relu(x):
    return np.maximum(0, x)

x = np.array([1., 2.])
W1 = np.array([[1., -2., 0.],
               [1., 1., -1.]]) # 2 numbers -> 3: expand
W2 = np.array([[1., 0.],
               [2., 1.],
               [0., 3.]]) # 3 numbers -> 2: project back

h = relu(x @ W1) # linear, then bend
y = h @ W2 # linear again
print(x @ W1, h, y)

y_lin = (x @ W1) @ W2 # the same layer without the bend
print(y_lin, x @ (W1 @ W2))
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. Linear: double in, double out · rewatch ▶ 0:08-0:27

1.1 B. $-3x$ triples and flips every input, so doubling in doubles out. For $2x + 1$, $f(1) = 3$ but $f(2) = 5$ (its line misses the origin); x^2 quadruples when x doubles; a constant never changes.

1.2 $W\mathbf{a} = (0.75, 1)$, $W\mathbf{b} = (-0.25, 1.75)$, $W(\mathbf{a} + \mathbf{b}) = W(1.5, 2) = (0.5, 2.75)$. Yes. Each output number is a row of W dotted with the vector, e.g. $1 \cdot 1 + (-0.5) \cdot 0.5 = 0.75$. And $(0.75, 1) + (-0.25, 1.75) = (0.5, 2.75)$, exactly $W(\mathbf{a} + \mathbf{b})$.

1.3 $W(2\mathbf{a}) = (1.5, 2)$ and $W(-\mathbf{a}) = (-0.75, -1)$. Scaling passes straight through a linear map: $W(2\mathbf{a}) = 2 \cdot W\mathbf{a}$, and scaling by -1 flips the output too.

1.4 True. $f(0) = 5 \cdot 0 = 0$. The same holds for any matrix: W times the zero vector is zero.

2. Linear layers collapse into one · rewatch ▶ 0:28-0:38

2.1 (a) $W_1\mathbf{x} = (3, 1)$, $W_2(W_1\mathbf{x}) = (6, 4)$. (b) W_2W_1 has rows $(2, 4)$ and $(1, 3)$; $(W_2W_1)\mathbf{x} = (6, 4)$. Same answer both ways: the two layers act exactly like the single matrix W_2W_1 .

2.2 Multiply by -3 . $3 \times (-2) \times 0.5 = -3$. However many multiplications you chain, they make one multiplication.

2.3 C. The whole stack multiplies out to one matrix, $W_{100} \cdots W_2W_1$, for every input.

2.4 False. They collapse into one matrix, so the network can do exactly what one layer can do, no more.

3. What a straight map can't do · rewatch ▶ 0:38-0:50

3.1 D. Stretching, rotating and shearing all keep straight lines straight. Bending is exactly what a linear map can't do.

3.2 No. The outside points sit at both ends. Any cut that puts 2 on one side leaves -2 on the other side, together with the inside points. (Concept 6 solves this with bends.)

3.3 False. The 50 layers collapse into one matrix (concept 2), with exactly the same limits.

4. The fix: a bend called ReLU · rewatch ▶ 0:51-1:08

4.1 $[4, 0, 0.5, 0]$. Positive numbers pass unchanged; negative ones become 0.

4.2 $\text{ReLU}(1) = 1$, but $\text{ReLU}(3) + \text{ReLU}(-2) = 3 + 0 = 3$. They differ, so ReLU is not linear. That is precisely what stops the layers collapsing into one matrix.

4.3 $\mathbf{x} \rightarrow W_1 \rightarrow \text{ReLU} \rightarrow W_2 \rightarrow \mathbf{y}$. Linear, then the bend, then linear again.

4.4 False. ReLU treats every number separately. The mixing is done by the matrices.

5. GELU: a smooth ReLU · rewatch ► 1:08-1:19

5.1 Both ≈ 0.004 . $3 - 2.996 = 0.004$ and $0 - (-0.004) = 0.004$. Far from zero, GELU is within a few thousandths of ReLU.

5.2 C. At ± 1 they differ by about 0.16 ($\text{relu}(1) = 1$, $\text{relu}(-1) = 0$); at ± 3 by only 0.004. The smooth curve replaces the corner at 0.

5.3 False. It has a small dip: its minimum is about -0.17 (near $x \approx -0.75$), and $\text{gelu}(-3) \approx -0.004$ is negative too, just tiny.

5.4 About $[-0.004, 0, 2.996]$, i.e. almost $[0, 0, 3]$. Each number is bent on its own, and this far from zero GELU acts almost exactly like ReLU.

6. Curves from many straight pieces · rewatch ► 1:20-1:27

6.1 $g = 1, 0, 0, 1$. Yes. Two hinges build a valley: 0 between -1 and 1 , rising outside. The outside points get 1 and the inside points 0, so “outside if $g(x) > 0.5$ ” works, which no single cut could do.

6.2 $0, 1, 0, -1$: up to a peak at $x = 1$, then down. The first hinge starts a rising line at 0; the second, from $x = 1$, subtracts twice the slope, so the line turns downward. Two pieces, one corner.

6.3 B. $2x - 3x = -x$. Sums of linear functions stay linear; corners and curves need bends.

7. A layer in code · rewatch ► 1:28-1:47

7.1 $W = W1 @ W2$. Without the bend the layer is linear after linear, so it collapses into their product (concept 2).

7.2 expand $\rightarrow$ bend with GELU $\rightarrow$ project back. Matrix $W1$ (expand), the bend, then matrix $W2$ (back to the original size).

7.3 (a) $x @ W1 = [3, 0, -2]$, $h = [3, 0, 0]$, $y = [3, 0]$ · (b) $[3, -6]$ twice · (c) ReLU zeroed the -2 . (a) Each entry of $x @ W1$ is x dotted with a column of $W1$: $1 \cdot 1 + 2 \cdot 1 = 3$, $1 \cdot (-2) + 2 \cdot 1 = 0$, $0 + 2 \cdot (-1) = -2$. ReLU turns -2 into 0, and $h @ W2 = 3 \cdot (1, 0) = (3, 0)$. (b) Without the bend the two matrices collapse into one: $W1 @ W2$ has rows $(-3, -2)$ and $(3, -2)$, and x times it gives $(3, -6)$, the same as doing the two steps. (c) In y_lin , the -2 in the third position adds $-2 \times (0, 3) = (0, -6)$. The bend removed it, so the layer is no longer a single matrix.